Identifiers:

An identifier is a name given to a variable, function, class or module. Identifiers may be one or more characters in the following format:

• Identifiers can be a combination of letters in lowercase (a to z) or uppercase (A to Z) or digits (0 to 9) or an underscore (_). Names like myCountry, other_1 and good_morning, all are valid examples. A Python identifier can begin with an alphabet (A – Z and a – z and _).

• An identifier cannot start with a digit but is allowed everywhere else. 1plus is invalid, but plus1 is perfectly fine.

• Keywords cannot be used as identifiers.

• One cannot use spaces and special symbols like !, @, #, $, % etc. as identifiers.

• Identifier can be of any length.

Keywords:

Keywords are a list of reserved words that have predefined meaning. Keywords are special vocabulary and cannot be used by programmers as identifiers for variables, functions, constants or with any identifier name. Attempting to use a keyword as an identifier name will cause an error.

List of Keywords in Python:

and	as	not	
assert	finally	or	
break	for	pass	
class	from	nonlocal	
continue	global	raise	
def	if	return	
del	import	try	
elif	in	while	
else	is	with	
except	lambda	yield	
False	True	None	

Statements and Expressions:

A statement is an instruction that the Python interpreter can execute. Python program consists of a sequence of statements. Statements are everything that can make up a line (or several lines) of Python code. For example, z = 1 is an assignment statement.

Expression is an arrangement of values and operators which are evaluated to make a new value. Expressions are statements as well. A value is the representation of some entity like a letter or a number that can be manipulated by a program. A single value >>> 20 or a single variable >>> z or a combination of variable, operator and value >>> z + 20 are all examples of expressions. An expression, when used in interactive mode is evaluated by the interpreter and result is displayed instantly. For example,

>>> 8 + 2

10

But the same expression when used in Python program does not show any output altogether. You need to explicitly print the result.

Data Types

The data stored in memory can be of many types. For example, a person's age is stored as a numeric value and his or her address is stored as alphanumeric characters. Python has various standard data types that are used to define the operations possible on them and the storage method for each of them.

Python has five standard data types:

- Number
- String
- List
- Tuple
- Dictionary

Number Data Type:

Number data types store numeric values. Number objects are created when you assign a value to them. For example:

var1 = 1

var2 = 10

You can also delete the reference to a number object by using the del statement. The syntax of the del statement is:

del var1[,var2[,var3[....,varN]]]]

You can delete a single object or multiple objects by using the del statement. For example:

del var

del var_a, var_b

Examples:
x = 1 # int

```
y = 2.8  # float
z = 1j   # complex
```

Another useful numeric data type for problem solvers is the complex number data type. A complex number is defined in Python using a real component + an imaginary component j. The letter j must be used to denote the imaginary component. Using the letter i to define a complex number returns an error in Python.

```
>>> comp = 4 + 2j
>>> type(comp)
<class 'complex'>

>>> comp2 = 4 + 2i
            ^
SyntaxError: invalid syntax
```

Imaginary numbers can be added to integers and floats.

```
>>> intgr = 3
>>> type(intgr)
<class 'int'>

>>> comp_sum = comp + intgr
>>> print(comp_sum)
(7+2j)

>>> flt = 2.1
>>> comp_sum = comp + flt
>>> print(comp_sum)
(6.1+2j)

#convert from int to float:
a = float(x)

#convert from float to int:
b = int(y)

#convert from int to complex:
c = complex(x)

print(a)
print(b)
print(c)

print(type(a))
```

```
print(type(b))
print(type(c))
```

Boolean Data Type:

Boolean data type is either True or False. In Python, boolean variables are defined by the True and False keywords.

```
>>> a = True
>>> type(a)
<class 'bool'>

>>> b = False
>>> type(b)
<class 'bool'>
```

Integers and Floats as Booleans

Integers and floating point numbers can be converted to the boolean data type using Python's bool() function. An int, float or complex number set to zero returns False. An integer, float or complex number set to any other number, positive or negative, returns True.

```
>>> zero_int = 0
>>> bool(zero_int)
False

>>> pos_int = 1
>>> bool(pos_int)
True

>>> neg_flt = -5.1
>>> bool(neg_flt)
True
```

Boolean Arithmetic

Boolean arithmetic is the arithmetic of true and false logic. A boolean or logical value can either be True or False. Boolean values can be manipulated and combined with boolean operators. Boolean operators in Python include and, or, and not.

The common boolean operators in Python are below:

or

and

not

== (equivalent)

!= (not equivalent)

In the code section below, two variables are assigned the boolean values True and False. Then these boolean values are combined and manipulated with boolean operators.

```
>>> A = True
>>> B = False

>>> A or B
True

>>> A and B
False

>>> not A
False

>>> not B
True

>>> A == B
False

>>> A != B
True
```

Boolean operators such as and, or, and not can be combined with parenthesis to make compound boolean expressions.

```
>>> C = False
>>> A or (C and B)
True
>>> (A and B) or C
False
```

When you compare two values, the expression is evaluated and Python returns the Boolean answer:

```
print(10 > 9)
print(10 == 9)
print(10 < 9)
```

String Data Type:

A string consists of a sequence of characters, which includes letters, numbers, punctuation marks and spaces. To represent strings, you can use a single quote, double quotes or triple quotes.

Basic String Operations

In Python, strings can also be concatenated using + sign and * operator is used to create a repeated sequence of strings.

1. >>> string_1 = "face"
2. >>> string_2 = "book"
3. >>> concatenated_string = string_1 + string_2
4. >>> concatenated_string
'facebook'
5. >>> concatenated_string_with_space = "Hi " + "There"
6. >>> concatenated_string_with_space
'Hi There'
7. >>> singer = 50 + "cent"
Traceback (most recent call last):
File "<stdin>", line 1, in <module>TypeError: unsupported operand type(s) for +: 'int' and 'str'
8. >>> singer = str(50) + "cent"
9. >>> singer
'50cent'
10. >>> repetition_of_string = "wow" * 5
11. >>> repetition_of_string
'wowwowwowwowwow'

You can check for the presence of a string in another string using **in** and **not in** membership operators. It returns either a Boolean True or False. The **in** operator evaluates to True if the string value in the left operand appears in the sequence of characters of string value in right operand. The **not in** operator evaluates to True if the string value in the left operand does not appear in the sequence of characters of string value in right operand.

1. >>> fruit_string = "apple is a fruit"
2. >>> fruit_sub_string = "apple"
3. >>> fruit_sub_string **in** fruit_string
True
4. >>> another_fruit_string = "orange"
5. >>> another_fruit_string **not in** fruit_string
True

String Comparison:

You can use (>, <, <=, >=, ==, !=) to compare two strings resulting in either Boolean Trueor False value. Python compares strings using ASCII value of the characters. For example,

```
1. >>> "january" == "jane"
False
2. >>> "january" != "jane"
True
3. >>> "january" < "jane"
False
4. >>> "january" > "jane"
True
5. >>> "january" <= "jane"
False
6. >>> "january" >= "jane"
True
7. >>> "filled" > ""
True
```

Built-In Functions:

There are many built-in functions for which a string can be passed as an argument

len(): The len() function calculates the number of characters in a string. The white space characters are also counted.

max(): The max() function returns a character having highest ASCII value.

min(): The min() function returns character having lowest ASCII value.

For example,

```
1. >>> count_characters = len("eskimos")
2. >>> count_characters
7
3. >>> max("axel")
'x'
4. >>> min("brad")
'a'
```

Accessing Characters in String by Index Number

Each character in the string occupies a position in the string. Each of the string's character corresponds to an index number. The first character is at index 0; the next character is at index 1, and so on. The length of a string is the number of characters in it. You can access each character in a string using a subscript operator i.e., a square bracket. Square brackets are used to perform indexing in a string to get the value at a specific index or position. This is also called subscript operator.

The syntax for accessing an individual character in a string is as shown below.
string_name[index]

where index is usually in the range of 0 to one less than the length of the string. The value of index should always be an integer and indicates the character to be accessed. For example,

```
1. >>> word_phrase = "be yourself"
2. >>> word_phrase[0]
'b'
3. >>> word_phrase[1]
'e'
4. >>> word_phrase[2]
' '
5. >>> word_phrase[3]
'y'
6. >>> word_phrase[10]
'f'
7. >>> word_phrase[11]
Traceback (most recent call last):
File "<stdin>", line 1, in <module>IndexError: string index out of range
```

You can also access individual characters in a string using negative indexing. If you have
a long string and want to access end characters in the string, then you can count backward
from the end of the string starting from an index number of −1. The negative index breakdown for the string “be yourself” assigned to word_phrase string variable is shown below.

```
1. >>> word_phrase[-1]
'f'
2. >>> word_phrase[-2]
'l'
```

String Slicing

The "slice" syntax is a handy way to refer to sub-parts of sequence of characters within an original string.

The syntax for string slicing is,
string_name[start:end[:step]]

With string slicing, you can access a sequence of characters by specifying a range of index numbers separated by a colon. String slicing returns a sequence of characters beginning atstart and extending up to but not including end. The start and end indexing values have to be integers. String slicing can be done using either positive or negative indexing.

```
1. >>> healthy_drink = "green tea"
```

```
2. >>> healthy_drink[0:3]
'gre'
3. >>> healthy_drink[:5]
'green'
4. >>> healthy_drink[6:]
'tea'
5. >>> healthy_drink[:]
'green tea'
6. >>> healthy_drink[4:4]
''
7. >>> healthy_drink[6:20]
'tea'
```

The negative index can be used to access individual characters in a string. Negative indexing starts with −1 index corresponding to the last character in the string and then the index decreases by one as we move to the left.

```
1. >>> healthy_drink[-3:-1]
'te'
2. >>> healthy_drink[6:-1]
'te'
```

Specifying Steps in Slice Operation

In the slice operation, a third argument called step which is an optional can be specified along with the start and end index numbers. This step refers to the number of characters that can be skipped after the start indexing character in the string. The default value of step is one. In the previous slicing examples, step is not specified and in its absence, a default value of one is used. For example,

```
1. >>> newspaper = "new york times"
2. >>> newspaper[0:12:4]
'ny'
3. >>> newspaper[::4]
'ny e'
```

Joining Strings Using join() Method

Strings can be joined with the join() string. The join() method provides a flexible way to concatenate strings.

The syntax of join() method is,string_name.join(sequence)

Here sequence can be string or list. If the sequence is a string, then join() function insertsstring_name between each character of the string sequence and returns the concatenated string. If the sequence is a list, then join() function inserts string_name between each item of list sequence and returns the concatenated string.

It should be noted that all the items in the list should be of string type.

```
1. >>> date_of_birth = ["17", "09", "1950"]
2. >>> ":".join(date_of_birth)
'17:09:1950'
3. >>> social_app = ["instagram", "is", "an", "photo", "sharing", "application"]
4. >>> " ".join(social_app)
'instagram is an photo sharing application'
5. >>> numbers = "123"
6. >>> characters = "amy"
7. >>> password = numbers.join(characters)
8. >>> password
'a123m123y'
```

Split Strings Using split() Method

The split() method returns a list of string items by breaking up the string using the delimiter string.

The syntax of split() method is,
string_name.split([separator [, maxsplit]])
Here separator is the delimiter string and is optional.

A given string is split into list of strings based on the specified separator. If the separator is not specified then whitespace is considered as the delimiter string to separate the strings. If maxsplit is given, at most maxsplit splits are done (thus, the list will have at most maxsplit + 1 items). If maxsplit is not specified or −1, then there is no limit on the number of splits.

```
1. >>> inventors = "edison, tesla, marconi, newton"
2. >>> inventors.split(",")
['edison', ' tesla', ' marconi', ' newton']
3. >>> watches = "rolex hublot cartier omega"
4. >>> watches.split()
['rolex', 'hublot', 'cartier', 'omega']
```

Strings Are Immutable

As strings are immutable, it cannot be modified. The characters in a string cannot be changed once a string value is assigned to string variable. However, you can assign different string values to the same string variable.

```
1. >>> immutable = "dollar"
2. >>> immutable[0] = "c"
Traceback (most recent call last):
File "<stdin>", line 1, in <module>TypeError: 'str' object does not support item assignment
3. >>> string_immutable = "c" + immutable[1:]
```

```
4. >>> string_immutable
'collar'
5. >>> immutable = "rollar"
6. >>> immutable
'rollar'
```

String Methods

You can get a list of all the methods associated with string by passing the str function to dir().

```
1. >>> dir(str)
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__getnewargs__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmod__', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'capitalize', 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isdecimal', 'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill']
```

For example,

```
1. >>> fact = "Abraham Lincoln was also a champion wrestler"
2. >>> fact.isalnum()
False
3. >>> "sailors".isalpha()
True
4. >>> "2018".isdigit()
True
5. >>> fact.islower()
False
6. >>> "TSAR BOMBA".isupper()
True
7. >>> "columbus".islower()
True
8. >>> warriors = "ancient gladiators were vegetarians"
9. >>> warriors.endswith("vegetarians")
True
10. >>> warriors.startswith("ancient")
True
11. >>> warriors.startswith("A")
False
12. >>> warriors.startswith("a")
True
13. >>> "cucumber".find("cu")
0
```

```
14. >>> "cucumber".find("um")
3
15. >>> "cucumber".find("xyz")
-1
16. >>> warriors.count("a")
5
17. >>> species = "charles darwin discovered galapagos tortoises"
18. >>> species.capitalize()
'Charles darwin discovered galapagos tortoises'
19. >>> species.title()
'Charles Darwin Discovered Galapagos Tortoises'
20. >>> "Tortoises".lower()
'tortoises'
21. >>> "galapagos".upper()
'GALAPAGOS'
22. >>> "Centennial Light".swapcase()
'cENTENNIAL lIGHT'
23. >>> "history does repeat".replace("does", "will")
'history will repeat'
24. >>> quote = " Never Stop Dreaming "
25. >>> quote.rstrip()
' Never Stop Dreaming'
26. >>> quote.lstrip()
'Never Stop Dreaming '
27. >>> quote.strip()
'Never Stop Dreaming'
28. >>> 'ab c\n\nde fg\rkl\r\n'.splitlines()
['ab c', '', 'de fg', 'kl']
29. >>> "scandinavian countries are rich".center(40)
'scandinavian countries are rich'
```

Input and Output in Python

In Python, we can simply use the print() function to print output. For example,

```
print('Python is powerful')
# Output: Python is powerful
```

Example 1: Python Print Statement

```
print('Good Morning!')
print('It is rainy today')
```

Example 2: Python print() with end Parameter

```
# print with end whitespace
print('Good Morning!', end= ' ')
print('It is rainy today')
```

Example 3: Python print() with sep parameter

```
print('New Year', 2023, 'See you soon!', sep= '. ')
```

Example: Print Python Variables and Literals

We can also use the print() function to print Python variables. For example,

```
number = -10.6
name = "course"

# print literals
print(5)

# print variables
print(number)
print(name)
```

Example: Print Concatenated Strings

We can also join two strings together inside the print() statement. For example,

```
print('course is ' + 'awesome.')
```

While programming, we might want to take the input from the user. In Python, we can use the input() function.

Syntax of input()

```
input(prompt)
```

Example: Python User Input

```
# using input() to take user input
num = input('Enter a number: ')

print('You Entered:', num)

print('Data type of num:', type(num))
```